

# UR3E MANAGER-BASED CUBE STACKING TASK

## MOTIVATION

The UR3e manager-based stack task is intended to train a simulated Universal Robots UR3e arm to stack three cubes on a table using a suction-style end effector. The task mirrors Isaac Lab's manager-based manipulation stack examples, but adapts the robot, TCP frame, and reward design to the local UR3e asset used in this repository.

The original Isaac Lab stack examples are well suited for imitation learning with robomimic when demonstration data is available. In this project, no demonstration dataset is assumed. For that reason, the task also exposes a 'skrl' PPO entry point and includes shaped rewards so that the policy can be trained from scratch.

The practical goal is to create a reusable sim training task for UR3e manipulation:

- \* validate the local UR3e USD asset in a contact-rich tabletop scenario;
- \* train a policy without manually collecting robomimic demonstrations;
- \* keep compatibility with Isaac Lab's manager-based environment structure;
- \* preserve a future path for behavior cloning if demonstrations are collected later.

## ENVIRONMENT DESCRIPTION

The environment is registered as two Gym tasks:

- \* 'Isaac-Robots-Stack-Cube-UR3e-Long-Suction-IK-Rel-v0'
- \* 'Isaac-Robots-Stack-Cube-UR3e-Short-Suction-IK-Rel-v0'

Both tasks use 'isaacsim.envs:ManagerBasedRLEnv' and the local manager-based UR3e stack configuration. The difference between the two tasks is the suction offset used for the end effector:

| Variant       | TCP body | Suction offset         |
|---------------|----------|------------------------|
| Long suction  | 'tool0'  | '0.22 m'               |
| Short suction | 'tool0'  | '0.1585 m' / '0.159 m' |

The scene contains:

- \* one UR3e articulation loaded from the local 'UR3E\_CFG';
- \* one Seattle lab table;
- \* three rigid cube objects: 'cube\_1', 'cube\_2', and 'cube\_3';
- \* a ground plane and dome light;
- \* a 'FrameTransformer' tracking the UR3e 'tool0' TCP frame;
- \* a surface gripper attached under '{ENV\_REGEX\_NS}/Robot/tool0/SurfaceGripper'.

Isaac Lab's surface gripper backend currently requires CPU simulation, so the environment sets both 'self.device' and 'self.sim.device' to 'cpu'.

## ACTION AND OBSERVATION SPACE

The task uses relative differential inverse kinematics for arm motion:

- \* arm action: relative end-effector pose command through 'DifferentialInverseKinematicsActionCfg';
- \* gripper action: binary surface-gripper command through 'SurfaceGripperBinaryActionCfg';
- \* IK body: 'tool0';
- \* IK method: damped least squares ('dls').

For PPO, the policy observation group is configured with 'concatenate\_terms = True'. This gives 'skrl' a flat observation tensor instead of the dictionary-style observation layout used by robomimic.

The policy observation contains:

- \* previous action;
- \* relative joint positions;
- \* relative joint velocities;
- \* stack object observation from Isaac Lab's stack MDP;
- \* cube positions in world frame;
- \* cube orientations in world frame;
- \* end-effector position;
- \* end-effector quaternion;
- \* gripper position.

## REWARD DESIGN

The PPO reward is intentionally shaped. Sparse success-only reward is unlikely to train efficiently for a three-cube contact task from scratch, especially with a suction gripper and no demonstrations. The reward therefore breaks the task into staged behaviors:

1. approach 'cube\_2';
2. lift 'cube\_2';
3. align 'cube\_2' over 'cube\_1';
4. stack 'cube\_2' on 'cube\_1';
5. approach 'cube\_3';
6. lift 'cube\_3';
7. align 'cube\_3' over 'cube\_2';
8. stack 'cube\_3' on 'cube\_2';
9. award a full-stack success bonus.

The current reward terms are:

Reward term   Weight   Purpose

'reach\_cube\_2'   '2.0'   Move the TCP close to 'cube\_2'.  
'lift\_cube\_2'   '4.0'   Lift 'cube\_2' above table height.  
'align\_cube\_2\_on\_1'   '4.0'   Align 'cube\_2' horizontally above 'cube\_1'.  
'stack\_cube\_2\_on\_1'   '10.0'   Reward a valid two-cube stack.  
'reach\_cube\_3'   '1.0'   Move the TCP close to 'cube\_3'.  
'lift\_cube\_3'   '3.0'   Lift 'cube\_3' above table height.  
'align\_cube\_3\_on\_2'   '5.0'   Align 'cube\_3' horizontally above 'cube\_2'.  
'stack\_cube\_3\_on\_2'   '15.0'   Reward placing the third cube on the stack.  
'full\_stack'   '30.0'   Reward the complete three-cube stack.

'action\_rate' '-0.005' Penalize abrupt action changes.  
'joint\_vel' '-0.0005' Penalize excessive joint velocity.

## Reward Formulas

The reaching reward uses a Gaussian distance shaping term:

$$\text{reward} = \exp(-(\text{distance} / \text{std})^2)$$

The lifting reward is clipped between '0' and '1':

$$\text{reward} = \text{clip}((\text{object\_z} - \text{table\_height}) / \text{lift\_height}, 0, 1)$$

The alignment reward combines horizontal alignment with a lifted gate:

$$\begin{aligned} \text{xy\_reward} &= \exp(-(\text{xy\_distance} / \text{xy\_std})^2) \\ \text{lifted\_gate} &= \text{clip}((\text{upper\_z} - \text{lower\_z}) / \text{height\_diff}, 0, 1) \\ \text{reward} &= \text{xy\_reward} * \text{lifted\_gate} \end{aligned}$$

The stack checks use cube-relative geometry instead of Franka finger-joint state. A pair is considered stacked when:

$$\begin{aligned} \text{xy\_distance}(\text{upper}, \text{lower}) &< 0.05 \text{ m} \\ \text{abs}((\text{upper\_z} - \text{lower\_z}) - 0.0468 \text{ m}) &< 0.01 \text{ m} \end{aligned}$$

The full success check requires both:

cube\_2 stacked on cube\_1  
cube\_3 stacked on cube\_2

## TERMINATION AND SUCCESS

The task keeps the upstream timeout and cube-dropping terminations. The original stack success condition is replaced with a local success check that does not assume a Franka gripper or finger joints. This is important because the UR3e task uses a surface gripper attached to 'tool0'.

An episode is successful when:

- \* 'cube\_2' is geometrically stacked on 'cube\_1';
- \* 'cube\_3' is geometrically stacked on 'cube\_2';
- \* the pairwise stack checks pass their horizontal and vertical thresholds.

## TRAINING PATH

The task supports two learning paths:

| Method       | Status                    | Use case                                               |
|--------------|---------------------------|--------------------------------------------------------|
| 'skrl' PPO   | Active                    | Train from scratch without demonstrations.             |
| robomimic BC | Registered for future use | Train from demonstrations once an HDF5 dataset exists. |

Recommended smoke test:

```
python scripts/zero_agent.py --task=Isaac-Robots-Stack-Cube-UR3e-Long-Suction-IK-Rel-v0
--device cpu --num_envs 1
```

Recommended first PPO run:

```
python scripts/skrl/train.py --task=Isaac-Robots-Stack-Cube-UR3e-Long-Suction-IK-Rel-v0
--device cpu --num_envs 16
```

The task is expected to be difficult for PPO. If learning is unstable, the recommended progression is:

1. train only the first two-cube stack behavior;
2. tune lift and alignment rewards;
3. re-enable the third cube;
4. increase parallel environments if CPU performance allows;
5. consider scripted or teleoperated demonstrations for robomimic if PPO plateaus.

## IMPLEMENTATION REFERENCES

Key local files:

- \* 'source/isaac\_robots/isaac\_robots/tasks/manager\_based/stack/config/ur3e\_gripper/\_\_init\_\_.py'
- \* 'source/isaac\_robots/isaac\_robots/tasks/manager\_based/stack/config/ur3e\_gripper/stack\_joint\_pos\_env\_cfg.py'
- \* 'source/isaac\_robots/isaac\_robots/tasks/manager\_based/stack/config/ur3e\_gripper/stack\_ik\_rel\_env\_cfg.py'
- \* 'source/isaac\_robots/isaac\_robots/tasks/manager\_based/stack/mdp/rewards.py'
- \* 'source/isaac\_robots/isaac\_robots/tasks/manager\_based/stack/config/ur3e\_gripper/agents/skrl\_ppo\_cfg.yaml'

External design reference:

- \* Isaac Lab manager-based manipulation stack task family.
- \* Isaac Lab surface gripper tutorial and CPU simulation requirement.